

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«ВЛАДИВОСТОКСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВВГУ»)
Институт информационных технологий и анализа данных
Кафедра информационных технологий и систем

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

по дисциплине «Информатика и основы программирования»

Лабораторная работа 3. Обработка строк

Студент		
гр. БИН-26-1	_____	Р.Е. Гаража
Преподаватель	_____	М.В. Водяницкий

1 Выполнение работы

1.1 Задание 1. Создание кода документа

В ходе выполнения задания предусмотрен ввод кода документа в формате AAA-NNNN-NNNN. Первая часть кода представлена тремя заглавными латинскими буквами и обозначает категорию, вторая содержит цифры года, третья включает четыре цифры номера.

Изображение алгоритма для решения первого задания 1.

```

1  doc_code = input("Введите код документа (AAA-NNNN-NNNN): ")
2
3  if len(doc_code) == 13 and doc_code[3] == '-' and doc_code[8] == '-':
4      category = doc_code[0:3]
5      year = doc_code[4:8]
6      number = doc_code[9:13]
7      reversed_number = number[::-1]
8
9      print(f"Категория: {category}")
10     print(f"Год: {year}")
11     print(f"Номер: {number}")
12     print(f"Обратный номер: {reversed_number}")
13 else:
14     print("Ошибка: неверный формат кода документа.")
15     print("Категория: 000")
16     print("Год: 0000")
17     print("Номер: 0000")
18     print(f"Обратный номер: 0000")

```

Рисунок 1 – Текст программы для первого задания

Требовалось с помощью срезов изменить внешний вид, год и номер, при этом сохранение полученных результатов не выполнялось. Дополнительно необходимо вывести номер в обратном порядке. Применение срезов позволяет извлекать нужные фрагменты строки по индексам, что упрощает и делает более наглядной обработку кода документа.

Изображение результата выполнения программы 2.

```

Введите код документа (AAA-NNNN-NNNN): RUS-2026-0042
Категория: RUS
Год: 2026
Номер: 0042
Обратный номер: 2400

```

Рисунок 2 – Результат выполнения программы для первого задания

На этом скриншоте показан результат обработки кода документа: программа корректно выделила категорию, год и номер, а также вывела номер в обратном порядке. Срезы строки позволили получить нужные части без изменения исходных данных.

1.2 Задание 2. Форматирование имени

В рамках выполнения задания пользователь вводит фамилию, имя и отчество одной строкой, при этом регистр букв может быть произвольным. Каждая часть строки состоит только из букв, между частями допускается один или несколько пробелов, а также возможны пробелы в начале и в конце строки.

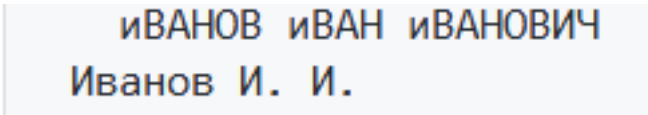
Изображение алгоритма для решения второго задания 3.

```
1  s = input()
2
3  parts = s.split()
4
5  surname = parts[0].capitalize()
6  name_initial = parts[1][0].upper() + "."
7  patronymic_initial = parts[2][0].upper() + "."
8
9  print(surname, name_initial, patronymic_initial)
```

Рисунок 3 – Текст программы для второго задания

Требовалось вывести фамилию, приведя первую букву к верхнему регистру, а остальные — к нижнему, после чего вывести инициалы имени и отчества в верхнем регистре. При обработке строки лишние пробелы необходимо игнорировать, а регистр букв привести к единому виду.

Изображение результата выполнения программы 4.



ИВАНОВ ИВАН ИВАНОВИЧ
Иванов И. И.

Рисунок 4 – Результат выполнения программы для второго задания

Здесь видно, как после обработки введённой строки фамилия приведена к нормальному регистру, а имя и отчество сокращены до инициалов. Лишние пробелы и смешанный регистр были успешно проигнорированы.

1.3 Задание 3. Очистка номера телефона

В рамках выполнения задания пользователь вводит номер телефона в формате +7 (NNN) NNN-NN-NN, где каждый символ N обозначает цифру от 0 до 9.

Изображение алгоритма для решения третьего задания 5.

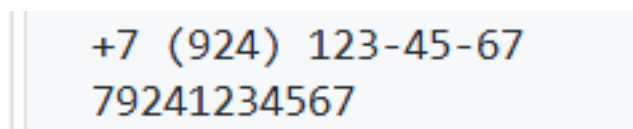
```

1 phone = input()
2
3 clean_phone = phone.replace('+', '').replace(' ', '').replace('(', '').replace(')', '').replace('-', '')
4
5 print(clean_phone)

```

Рисунок 5 – Текст программы для третьего задания

Требовалось удалить из введённой строки символы «+», пробелы, круглые скобки и дефисы. В результате должна остаться только последовательность цифр, пригодная для дальнейшей обработки или хранения. Изображение результата выполнения программы 6.



```

+7 (924) 123-45-67
79241234567

```

Рисунок 6 – Результат выполнения программы для третьего задания

Результат работы программы подтверждает, что из номера телефона удалены все лишние символы: плюс, пробелы, скобки и дефисы. В строке остались только цифры, что удобно для дальнейшей обработки.

1.4 Задание 4. Разбор записей о поезде

В рамках выполнения задания пользователь вводит сведения о поезде одной строкой, поля которой разделены символом «;» в следующем порядке: номер, пункт отправления, пункт назначения, время, цена. Все пять полей должны быть непустыми, не содержать символа «;» и пробелов по краям; внутри названий городов пробелы допустимы. Время задаётся в формате HH:MM, цена — неотрицательное число, в котором десятичным разделителем является точка.

Изображение алгоритма для решения четвертого задания 7.

```

1  data = input()
2
3  parts = data.split(';')
4
5  train_number = parts[0]
6  from_city = parts[1]
7  to_city = parts[2]
8  departure_time = parts[3]
9  price = float(parts[4])
10
11 print(f"Поезд: {train_number}")
12 print(f"Маршрут: {from_city} - {to_city}")
13 print(f"Отправление: {departure_time}")
14 print(f"Цена: {price:.2f} руб")

```

Рисунок 7 – Текст программы для четвертого задания

Требовалось разобрать введенную строку на отдельные значения и вывести официальную запись. Цена при этом выводится с двумя цифрами после десятичной точки. Для разбора строки удобно использовать метод `split()` с разделителем «;», а для форматирования цены — форматированную строку. Изображение результата выполнения программы 8.

```

Т-18;Владивосток;Хабаровск;08:45;1850.5
Поезд: Т-18
Маршрут: Владивосток - Хабаровск
Отправление: 08:45
Цена: 1850.50 руб

```

Рисунок 8 – Результат выполнения программы для четвертого задания

В консоли выведена официальная запись о поезде с номером, маршрутом, временем отправления и ценой. Цена представлена с двумя знаками после десятичной точки, как и требовалось в задании.

1.5 Задание 5. Проверка идентификатора

В ходе выполнения задания необходимо проверить идентификатор, вводимый пользователем в виде одной строки. Строка может быть пустой или содержать пробелы. Проверка осуществляется над исходным текстом без удаления, замены или иного изменения символов.

Требуется вывести длину строки и результаты следующих проверок: состоит ли строка только из букв, только из цифр, является ли она буквенно-цифровой, а также содержит ли она дефис. Буквенно-цифровой считается непустая строка, каждый символ которой является буквой или цифрой.

Изображение алгоритма для решения пятого задания 9.

```
1  text = input()
2
3  length = len(text)
4  only_letters = text.isalpha()
5  only_digits = text.isdigit()
6  alphanumeric = text.isalnum()
7  has_dash = '-' in text
8
9  print(f"Длина: {length}")
10 print(f"Только буквы: {only_letters}")
11 print(f"Только цифры: {only_digits}")
12 print(f"Буквенно-цифровая: {alphanumeric}")
13 print(f"Содержит дефис: {has_dash}")
```

Рисунок 9 – Текст программы для пятого задания

Для первых трёх проверок используются методы `isalpha()` и `isdigit()`, для четвёртой — `isalnum()`. Если строка пустая, её длина равна 0, а все четыре проверки возвращают `False`. Такой подход позволяет оценить состав строки без её модификации.

Изображение результата выполнения программы 10.

```
AB-204
Длина: 6
Только буквы: False
Только цифры: False
Буквенно-цифровая: False
Содержит дефис: True
```

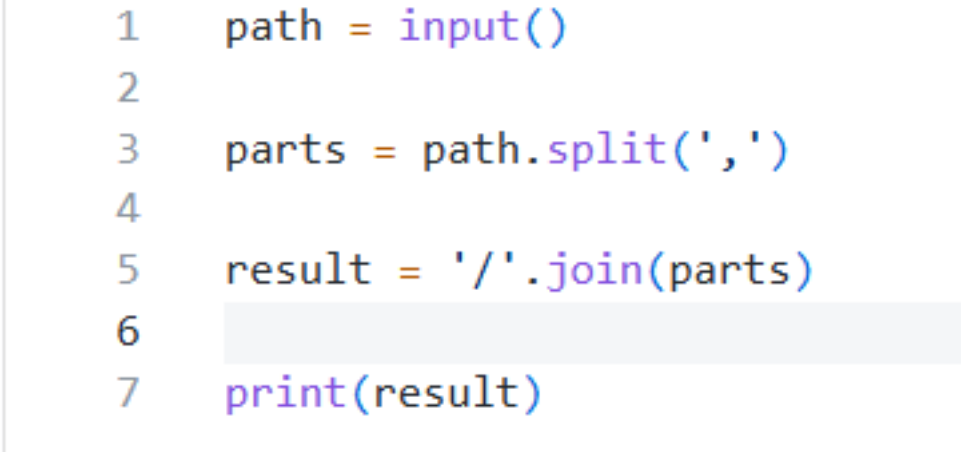
Рисунок 10 – Результат выполнения программы для пятого задания

На скриншоте отражены длина строки и результаты проверок на состав символов. Исходный идентификатор не изменялся, а все проверки выполнены строковыми методами.

1.6 Задание 6. Формирование пути (для отчёта)

В рамках выполнения задания пользователь вводит строку, содержащую ровно три непустые части пути, разделённые запятыми без пробелов. Части пути не должны содержать символы / и \.

Изображение алгоритма для решения шестого задания 11.

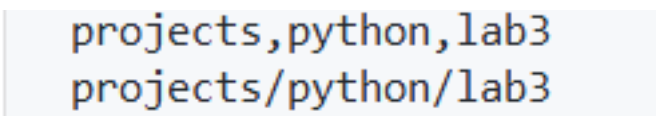
A screenshot of a code editor showing a Python script. The code consists of seven lines: 1. path = input(), 2. (empty line), 3. parts = path.split(','), 4. (empty line), 5. result = '/'.join(parts), 6. (empty line), 7. print(result). The code is color-coded: keywords like input, split, join, and print are in blue, and variables like path, parts, and result are in black. The line numbers 1 through 7 are on the left side of the code block.

```
1 path = input()
2
3 parts = path.split(',')
4
5 result = '/'.join(parts)
6
7 print(result)
```

Рисунок 11 – Текст программы для шестого задания

Необходимо разделить введённую строку на составляющие с помощью метода `split()`, а затем объединить полученные элементы символом `/` с использованием метода `join()`. В результате выполнения программы должен быть получен корректный относительный путь, состоящий из трёх уровней.

Изображение результата выполнения программы 12.

A screenshot of a terminal window showing the output of the program. The first line shows the input string 'projects,python,lab3' in blue. The second line shows the resulting path 'projects/python/lab3' in black.

```
projects,python,lab3
projects/python/lab3
```

Рисунок 12 – Результат выполнения программы для шестого задания

Здесь показан итоговый путь, собранный из трёх частей с разделителем `/`. Методы `split()` и `join()` позволили разделить строку и соединить её элементы в корректный относительный путь